



UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA
DEPARTAMENTO DE TECNOLOGIA
ÁREA DE ELETRÔNICA
TEC 499 - MI SISTEMAS DIGITAIS

Problema – 2026.1

1. Tema

Implementação de um algoritmo em um circuito especializado em FPGA com desenvolvimento de driver Linux em Assembly e de uma aplicação em linguagem C para utilizar o algoritmo implementado na FPGA.

2. Objetivo de Aprendizagem

O objetivo acadêmico é conectar a teoria de Arquitetura de Computadores (datapath, controle, paralelismo, memórias, I/O mapeado em memória, latência/throughput) com a prática de aceleração por hardware e interface Sistema Operacional/Hardware.

Espera-se que ao final deste componente o/a discente seja capaz de:

- Aplicar conhecimentos de circuitos digitais e arquiteturas de computadores para desenvolver uma solução completa de hardware e software;
- Entender conceitos de Implementação de drivers de software;
- Entender dificuldades tecnológicas no desenvolvimento de sistemas digitais de propósito específico;
- Compreender a interface hardware-software;
- Compreender o mapeamento de memória em uma arquitetura ARM;
- Utilizar a interface de conexão entre HPS e FPGA da DE1-SoC;
- Programar em assembly para um processador com arquitetura ARM;
- Entender políticas de gerenciamento de sistema operacional Linux em arquitetura ARM;
- Compreender os princípios básicos da arquitetura da plataforma DE1-SoC;
- Utilizar as interfaces disponíveis na placa DE1-SoC;
- Entender o conceito de link-edição com acoplamento de diversos módulos objeto;
- Entender como uma aplicação em C pode ser usada para acessar funcionalidade de um dispositivo de hardware dedicado.

3. Desafio

Você faz parte de uma equipe que precisa construir um **classificador de imagens de dígitos numéricos**, que será embarcado em um SoC (System on Chip) heterogêneo (ARM + FPGA). O sistema completo será composto por:

1. **Núcleo classificador de imagens em FPGA (Verilog)**
Um co-processador com conjunto próprio de instruções (ISA), o qual implementa a inferência (classificação da imagem) de uma rede neural baseada em Extreme Learning Machine (ELM).
2. **Driver em Linux (ARM) em Assembly**
Uma API (sistema operacional Linux) para controlar o co-processador via MMIO (Memory-Mapped I/O).
3. **Aplicação em C (usuário)**
Uma aplicação em linguagem C para ler uma imagem com 28×28 pixels em escala de cinza no formato PNG, envia ao driver, receber o dígito previsto (0..9) e imprimi-lo na tela.

3.1 Escopo da rede neural ELM

Não será necessário treinar a rede ELM (Extreme Learning Machine), ela será tratada como uma sequência de quatro estágios (Fig. 1):

1. **Leitura do vetor de entrada representando a imagens (784 bytes)**
2. **Processamento da camada oculta:**
 - $h = \text{activation}(W_{in} \cdot x + b)$
3. **Processamento da camada de saída:**
 - $y = \beta \cdot h$
4. **Cômputo da Predição:**
 - $\text{pred} = \text{argmax}(y) \rightarrow \text{retorna } 0..9$

O modelo base (parâmetros W_{in} , b , β) será fornecido através de um conjunto de arquivos.

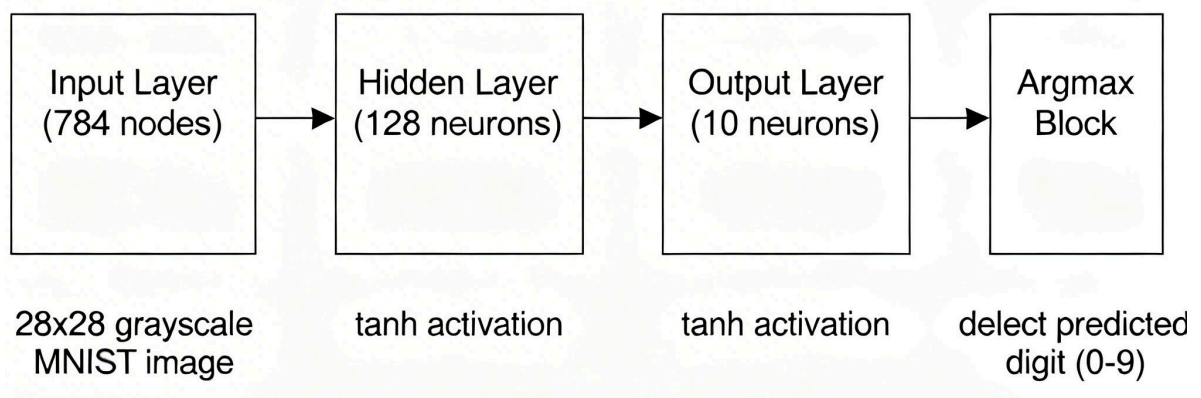


Figura 1: Diagrama geral do motor de inferência na ELM

3.2 Arquitetura do conjunto de instruções (ISA)

O co-processador deverá ter o seguinte conjunto mínimo de instruções (todas com 32-bit):

- 1) **STORE_IMG**: Armazena a imagem na memória para classificação;

- 2) **STORE_WEIGHTS**: Armazena os pesos (W) na memória;
- 3) **STORE_BIAS**: Armazena os vieses (b) na memória;
- 4) **START**: Inicia o processamento (classificação) da imagem na memória;
- 5) **STATUS**: devolve o estado do processamento e o valor predito, de acordo com as seguintes possibilidades:
 - a) **BUSY**: em processamento;
 - b) **DONE**: processamento concluído;
 - c) **ERROR**: erro no processamento;
 - d) Além do estado do processamento, a instrução **STATUS** devolve também o resultado da inferência como um número inteiro $[0,9]$ codificado em 4-bit.

4. Requisitos

4.1 Entrada/Saída

- A entrada é uma imagem escala de cinza com 28×28 pixels, 8 bits/pixel $[0, 255]$ num arquivo formato PNG com 784 Bytes. Cada imagem representa apenas um dígito (0..9);
- A saída é um número inteiro ***pred*** no intervalo $[0, 9]$

4.2 Co-processador (Núcleo ELM — Verilog) (Para o Marco 1)

- Implementar inferência ELM com pesos fornecidos.
- A arquitetura deve ser sequencial
- Deve haver:
 - FSM de controle
 - datapath MAC (multiplica-acumula)
 - ativação aproximada (LUT ou piecewise linear)
 - argmax final
 - memórias para armazenamento dos dados
 - banco de registradores
- Valores devem ser representados em ponto fixo no formato **Q4.12**.
- Pesos devem ser armazenados em blocos de memória.
- Deve haver uma estratégia clara para armazenamento e acesso a W_{in} , b e β

4.3 Driver (Linux ARM — Assembly) (Para o Marco 2)

Implementar um driver em linguagem Assembly ARM que permita à aplicação:

- inicializar o hardware;
- enviar a imagem;
- enviar os pesos e o bias;
- iniciar inferência;
- aguardar finalização (polling ou interrupção);
- ler resultados e métricas.

Você deve entregar:

- código do driver (e build);
- documentação da API (protótipos, semântica, erros);
- teste mínimo: classificar uma imagem conhecida repetidas vezes sem falhas.

O modelo (tipo) de driver será definido pelo tutor (módulo de kernel ou userspace via /dev/mem). O enunciado exige MMIO e API consistente independentemente do tipo.

4.4 Aplicação (C — usuário) (Para o Marco 3)

A aplicação final deve:

- integrar ao projeto o IP-Core VGA disponível em:
https://github.com/DestinyWolf/Problema_SD_2026_1?authuser=0
- ter 3 modos de operação:
 - modo de inferência a partir de imagem oriunda de um arquivo;
 - modo de inferência a partir de imagem desenhada na tela;
 - modo de validação/benchmark.
- desenhar uma imagem usando um mouse e o IP-Core VGA;
- receber caminho para a imagem e parâmetros (via linha de comando);
- exibir a imagem a ser inferida usando o IP-Core VGA;
- enviar imagem ao coprocessador;
- obter e imprimir o número predito, através da interface do programa desenvolvida em modo texto;
- ter modo de validação/benchmark, através da interface do programa desenvolvida em modo texto:
 - rodar N imagens e computar:
 - acurácia (%);
 - latência média e desvio;
 - throughput (imagens/s);
- salvar um log simples (CSV recomendado);

5. Avaliação e Nota

Para avaliar o envolvimento do grupo nas discussões e na apresentação, o tutor poderá fazer perguntas variadas a qualquer membro, tanto nas sessões tutoriais quanto na apresentação. O estudante que não comparecer, ou se atrasar, no dia da sessão de apresentação, terá automaticamente nota 0,0 (zero) nesta avaliação, excetuando-se as condições que permitem 2ª chamada de avaliações, conforme regulamento do curso.

A média final do MI será composta pela média ponderada das notas parciais de acordo com a fórmula a seguir (Equação 1), em que N1, N2 e N3 são as notas dos marcos 1, 2 e 3, respectivamente.:

$$Média = (N1 \times 4 + N2 \times 3 + N3 \times 3) \div 10 \quad (\text{Eq. 1})$$

Cada nota parcial é computada a partir da média ponderada dos pontos de avaliação descritos na Tabela 1, e corresponde a um marco de avaliação em que as equipes apresentarão suas soluções no laboratório e enviarão o link do repositório com os códigos e o READ.ME para o tutor. As datas dos marcos de avaliação estão definidas no calendário que será disponibilizado pelo tutor.

Tabela 1: Barema das notas parciais		
Critério	Crítérios para a nota	Peso
Desempenho Individual	Participação individual nas sessões tutoriais, de acordo com o interesse e entendimento demonstrados pelo aluno, assim como sua assiduidade, pontualidade e contribuição nas discussões. Essa nota inclui o desempenho do estudante na apresentação do problema no laboratório.	3
Projeto e Códigos	Qualidade do código fonte (organização e comentários), e execução correta dos códigos binários de acordo com testes de validação que explorem as situações de uso, atendimento a todos requisitos e funcionalidades.	4
Documentação	Ver barema da documentação na Tabela 2.	3

Tabela 2: Barema da documentação	
Descrição	Peso
Qualidade da Redação (Ortografia e Gramática)	1
Organização dos Tópicos	1
Definição do Problema	1
Teoria	1
Descrição da Solução	2
Explicação dos Testes	2
Análise dos Resultados	2

6. Critérios para avaliação do projeto

Para auxiliar na organização da informação do repositório, seguem abaixo os conteúdos mínimos necessários para cada marco de avaliação.

Marco 1 — Co-processador em FPGA + Simulação

Objetivo: construir e validar o IP do motor de inferência ELM em Verilog.

Entregas obrigatórias

- RTL Verilog do IP `elm_accel`
- Testbench com ao menos `K` vetores de teste (fornecidos pelo professor) comparando com golden model
- Repositório GitHub contendo:
 - diagrama de blocos (datapath + FSM)
 - uso de recursos FPGA (LUT/FF/DSP/BRAM)
 - mapa preliminar de registradores (CTRL/STATUS/IMG/RESULT/CYCLES)
 - scripts para automação dos testes;
 - README com detalhamento da sua solução (ver abaixo)
- Conteúdo mínimo do arquivo README do projeto:
 - Levantamento de requisitos;
 - Detalhamento dos software (com respectivas versões) usados no trabalho, incluindo softwares básicos;
 - Especificação dos hardwares usados nos testes;
 - Descrição detalhada do processo para instalação e configuração de ambiente para uso da solução;
 - Descrição detalhada dos testes de funcionamento e inclusão dos softwares e códigos usados para automação dos testes.
 - Análise dos resultados alcançados.

Marco 2 — Integração HW↔Linux via Driver (Assembly)

Objetivo: integrar o IP ao HPS e controlar a inferência via MMIO.

Entregas obrigatórias

- Projeto Quartus integrando HPS↔FPGA (bridges) com o IP mapeado
- Driver Linux com:
 - rotinas críticas em Assembly ARM
 - API definida (ex.: `open/ioctl/read/write` ou equivalente)
- Demonstração:
 - enviar 1 imagem fixa e obter classificação correta repetidamente (estabilidade)
- Repositório GitHub contendo:
 - código assembly comentado;
 - scripts para automação dos testes;
 - README com detalhamento da sua solução (ver abaixo)
- Conteúdo mínimo do arquivo README do projeto:
 - Levantamento de requisitos;
 - Detalhamento dos software (com respectivas versões) usados no trabalho, incluindo softwares básicos;

- Descrição detalhada do processo para instalação e configuração de ambiente para uso da solução;
- Descrição detalhada dos testes de funcionamento e inclusão dos softwares e códigos usados para automação dos testes.
- Análise dos resultados alcançados.

Marco 3 — Aplicação C + Validação Completa + Métricas

Objetivo: entregar o sistema final utilizável e mensurado.

Entregas obrigatórias

- Aplicação em C com CLI (classificação individual + dataset + benchmark)
- Demonstração final:
 - rodar dataset de teste e apresentar métricas
- Conteúdo mínimo do arquivo READ.ME do projeto:
 - Levantamento de requisitos;
 - Detalhamento dos software (com respectivas versões) usados no trabalho, incluindo softwares básicos;
 - Descrição detalhada do processo para instalação e configuração de ambiente para uso da solução;
 - Descrição detalhada dos testes de funcionamento e inclusão dos softwares e códigos usados para automação dos testes;
 - Arquitetura final (hardware e software);
 - Resultados de acurácia e desempenho;
 - Análise dos resultados alcançados, descrevendo gargalos encontrados e melhorias tentadas.

6. Calendário

O calendário com a programação das aulas e datas de entrega será entregue pelo tutor no primeiro dia de aula.

Links úteis e Referências

- Site do Laboratório de Eletrônica Digital e Sistemas (LEDS):
<https://sites.google.com/uefs.br/ltec3-leds>
- FPGA Academy: <https://fpgacademy.org/boards.html>
- Accelerating Extreme Learning Machine on FPGA Hardware Implementation of Given Rotation - QRD - <https://publisher.uthm.edu.my/ojs/index.php/ijie/article/view/4431>
- A máquina de aprendizado extremo - ELM -
<https://computacaointeligente.com.br/algoritmos/maquina-de-aprendizado-extremo/>
- Extreme learning machine: algorithm, theory and applications -
https://www.researchgate.net/publication/257512921_Extreme_learning_machine_algorithm_theory_and_applications